RETO 1

El modelo de información de un cine

Cine «Las Palmas»

Asignatura: Arquitectura Orientada a Servicios
Unidad 3: Semántica

Objetivo

Diseñar el modelo de información del cine Las Palmas mediante clases, atributos, asociaciones, cardinalidades e identificadores, y representar una función en JSON y un boleto en XML.

Alcance

El sistema considera películas, salas, funciones, boletos y clientes. El cine cuenta con seis salas. Cada función relaciona una película con una sala en una fecha y hora determinadas y establece el precio del boleto. Los clientes registrados acumulan puntos por sus compras.

Entregables

Diagrama UML, tabla de identificadores, restricciones semánticas, ejemplo JSON anidado, ejemplo XML con referencias por identificador y una implementación funcional de apoyo en Java con Spring Boot.

1. Clases y atributos

Las clases se identifican a partir de los conceptos principales del caso. Los tipos se eligieron de acuerdo con la información que almacena cada atributo.

Clase	Atributos y tipos
Pelicula	id: Long; titulo: String; clasificacion: Clasificacion; duracion: Integer (minutos); genero: String
Sala	id: Long; numero: Integer; capacidad: Integer
Funcion	id: Long; fechaHora: LocalDateTime; precio: Decimal; pelicula: Pelicula; sala: Sala
Boleto	id: Long; folio: String; fila: String; numero: Integer; funcion: Funcion; cliente: Cliente opcional
Cliente	id: Long; nombre: String; email: String; registrado: Boolean; puntos: Integer

Tipo enumerado

Clasificacion solo admite A, B, B15 y C. Se representa como enumeración y se almacena como texto.

2. Asociaciones y cardinalidades

Asociación	Cardinalidad	Justificación
Pelicula–Funcion	1 a 0..*	Una película puede programarse en varias funciones; cada función proyecta una película.
Sala–Funcion	1 a 0..*	Una sala puede albergar distintas funciones en horarios diferentes; cada función ocupa una sala.
Funcion–Boleto	1 a 0..*	Una función puede vender varios boletos; cada boleto corresponde a una función.
Cliente–Boleto	0..1 a 0..*	Un boleto puede asociarse con cero o un cliente; un cliente puede comprar varios.

3. Diagrama de clases UML

El archivo editable documentacion/diagrama-clases.puml contiene el diagrama UML con las clases, atributos y multiplicidades.

```
Pelicula "1" -------- "0..*" Funcion
Sala "1" ------------ "0..*" Funcion
Funcion "1" --------- "0..*" Boleto
Cliente "0..1" ------- "0..*" Boleto

Pelicula: id, titulo, clasificacion, duracion, genero
Sala: id, numero, capacidad
Funcion: id, fechaHora, precio
Boleto: id, folio, fila, numero
Cliente: id, nombre, email, registrado, puntos
Clasificacion: A | B | B15 | C
```

4. Identificadores

Clase	Identificador	Tipo y motivo
Pelicula	id	Sustituto: se genera automáticamente y no depende del título.
Sala	id	Sustituto; numero es una clave candidata natural y única.
Funcion	id	Sustituto: sala y fecha/hora pueden cambiar.
Boleto	folio	Natural de negocio; id se conserva como sustituto interno.
Cliente	id	Sustituto; email es una clave candidata natural que puede cambiar.

5. Restricciones semánticas

1. La clasificación solo admite A, B, B15 y C.

2. La duración de una película debe ser un entero mayor que cero y se expresa en minutos.

3. El precio de una función debe ser mayor que cero.

4. En una función no puede venderse dos veces la combinación de fila y número de asiento.

5. Únicamente los clientes registrados acumulan puntos por compra.

6. Una sala no puede tener funciones cuyos horarios se traslapen, considerando la duración de cada película.

6. Representación JSON de una función

La función contiene la fecha y hora, el precio y los objetos de película y sala anidados.

```
{
  "fechaHora": "2026-11-05T18:30:00-06:00",
  "precio": 85.00,
  "pelicula": {
    "id": 1,
    "titulo": "El viaje inesperado",
    "clasificacion": "B",
    "duracion": 110,
    "genero": "Aventura"
  },
  "sala": {
    "id": 2,
    "numero": 2,
    "capacidad": 100
  }
}
```

El archivo ejemplos/funcion.schema.json valida los campos, tipos, clasificación, duración y precio.

7. Representación XML de un boleto

El boleto conserva sus atributos y utiliza referencias por identificador para relacionarse con la función y el cliente, sin duplicar los objetos completos.

```
<?xml version="1.0" encoding="UTF-8"?>
<boleto>
    <folio>BOL-8F2A91C4</folio>
    <fila>C</fila>
    <numero>12</numero>
    <funcion ref="1"/>
    <cliente ref="3"/>
</boleto>
```

El archivo ejemplos/boleto.xsd valida la estructura XML, el formato de fila y que las referencias sean identificadores positivos.

8. Implementación del proyecto

La implementación utiliza Java 17, Spring Boot, Spring Data JPA y H2 como almacenamiento local. Se mantiene la separación de modelos, repositorios, servicios y controladores del código base. Se inicializan seis salas al primer arranque.

Se implementan validaciones de películas y salas, programación de funciones sin traslapes, compra de boletos, bloqueo de asientos duplicados y acumulación de puntos para clientes registrados. Los errores de negocio se devuelven con respuestas HTTP descriptivas.

Ejecución: abrir una terminal en la carpeta del proyecto y ejecutar `mvn spring-boot:run`. La API queda disponible en `http://localhost:8080`. Los endpoints y ejemplos adicionales están en README.md.

Archivos: README.md, documentacion/diagrama-clases.puml, ejemplos/funcion.json, ejemplos/boleto.xml, ejemplos/funcion.schema.json y ejemplos/boleto.xsd.